

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: А. А. Кухтичев
Группа: М8О-206Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Лабораторная работа №2

Задача: Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь.

Вариант структуры данных: PATRICIA.

Вариант ключа: регистронезависимая последовательность букв английского алфавита длиной не более 256 символов.

Вариант значения: числа от 0 до $2^{64} - 1$.

1 Описание

Требуется реализовать структуру данных PATRICIA. Основная ее идея заключается в том, чтобы при обходе вершин сравнивать не строки целиком, а один нужный бит.

Пользуясь информацией из [2], мы понимаем, что сложность поиска будет равна $\Theta(h)$, где h - высота дерева, так как мы сравниваем только один бит в нашей строке, и далее идем в нужного сына. Вставка в данной структуре данных работает за $\Theta(2h) = \Theta(h)$, потому что для данного алгоритма нам нужно сначала найти «похожий» элемент, а потом место для вставки. Для удаления сложность так же будет $\Theta(h)$, для алгоритма надо четыре раза применить поиск в дереве. Высота h в дереве приблизительно равна $\log(n)$.

Также требуется реализовать обратку строк вида «+ word num», «- word», «word», означающие добавление, удаление и поиск слова в дереве соответственно.

2 Исходный код

Для каждого узла в дереве у меня хранится динамический массив `char`, номер бита `int`, данные в `unsigned long long` и указатели на двоих сыновей. В строке мы будем хранить изначальный ключ, а не его битовое представление. Таким образом мы сократим затраты памяти. Во время алгоритмов поиска, вставки и удаления на каждой итерации будем осуществлять побитовый сдвиг текущего ключа, чтобы узнать нужный бит.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 typedef struct Patricia {
6     unsigned long long data;
7     int bit;
8     char* word;
9
10    struct Patricia* left;
11    struct Patricia* right;
12 } Patricia;
13 }
```

patricia.c	
Patricia* createPatricia(unsigned long long value, char* str);	Создание дерева
Patricia* search(Patricia* node, char* str, int lastbit);	Поиск в дереве по ключу str
Patricia × searchNodeForInsert(Patricia × node, Patricia searched, int searched_bit, Patricia parent, int lastbit);	Вспомогательная функция для вставки в дерево
void insertToPatricia(Patricia × node, char str, unsigned long long value, int <i>ok_{rot}</i>);	Алгоритм вставки в дерево
Patricia* searchBackLinkToNode(Patricia × node, Patricia searched, Patricia parent, int lastbit);	Вспомогательная функция для удаления из дерева, ищет обратную ссылку на переданный узел

Patricia× searchDirectLinkToNode(Patricia node, Patricia searched, Patricia parent, int lastbit);	Вспомогательная функция для удаления из дерева, ищет прямую ссылку на переданный узел
void removeNode(Patricia root, char str);	Алгоритм удаления из дерева
void printOneNode(Patricia node);	Вспомогательная функция для вывода дерева
void printPatriciaInAddress(Patricia root, int last_bit);	Функция вывода дерева со всей информацией о каждом узле
void deletePatricia(Patricia node, int last_bit);	Функция удаления дерева

Для реализации ввода я использовал цикл while. В каждой итерации цикла я читаю введенную строку, и проверяю, подходит ли ее формат под чтение, загрузку, удаление узла или загрузку и сохранение в файл. Чтобы записать дерево в файл, я сначала открываю файл на запись в бинарном виде, прохожусь по каждому узлу в дереве и записываю ключ и значение с помощью команды fwrite(). Для чтения из файла я также открываю файл на чтение в бинарном виде и читаю по очереди размер ключа, ключ и значение с помощью команды fread().

```

1 | int main() {
2 |     unsigned long long val;
3 |     Patricia* root = NULL;
4 |     int flag_have_tree = 0;
5 |     char input[1400];
6 |     char word[500];
7 |     char val_str[65];
8 |     char path[256];
9 |     while (fgets(input, sizeof(input), stdin)) {
10 |         if (input[0] == '+') {
11 |             sscanf(input, "+ %s %s", word, val_str);
12 |             val = strtoull(val_str, NULL, 10);
13 |             toLowercase(word);
14 |             char* binary_word = word;
15 |             if (flag_have_tree == 1) {
16 |                 insertToPatricia(root, binary_word, val, 1);
17 |             }
18 |             else {
19 |                 root = createPatricia(val, binary_word);
20 |                 flag_have_tree = 1;
21 |                 printf("OK\n");
22 |             }
23 |         }

```

```

24     else if (input[0] == '-') {
25         if (root != NULL) {
26             sscanf(input, "- %s\n", word);
27             toLowercase(word);
28             char* binary_word = word;
29             if (root->bit == -1 && root->left == root && strcmp(binary_word, root->
30                 word) == 0) {
31                 free(root->word);
32                 free(root);
33                 root = NULL;
34                 printf("OK\n");
35                 flag_have_tree = 0;
36             }
37             else
38                 removeNode(root, binary_word);
39         }
40         else {
41             printf("NoSuchWord\n");
42         }
43     }
44     else if (input[0] == '!') {
45         if (input[2] == 'S') {
46             sscanf(input, "! Save %s", path);
47             FILE* file = fopen(path, "wb");
48             writePatriciaToFile(file, root, -2);
49             fclose(file);
50             printf("OK\n");
51         }
52         if (input[2] == 'L') {
53             int checker_in_load;
54             sscanf(input, "! Load %s", path);
55             Patricia * new_root = NULL;
56             FILE* file = fopen(path, "rb");
57             new_root = readPatriciaFromFile(file, &checker_in_load);
58             fclose(file);
59             if (new_root == NULL && checker_in_load == 0){
60                 printf("ERROR: incorrect data format");
61             }
62             else{
63                 deletePatricia(root, -2);
64                 root = new_root;
65                 printf("OK\n");
66                 flag_have_tree = 1;
67             }
68             if (new_root == NULL) {
69                 flag_have_tree = 0;
70             }
71         }
72     }

```

```

72     }
73     else {
74         sscanf(input, "%s\n", word);
75         toLowercase(word);
76         char* binary_word = word;
77         Patricia* node = search(root, binary_word, -2);
78         if (node == NULL) {
79             printf("NoSuchWord\n");
80         }
81         else if (strcmp(binary_word, node->word) == 0) {
82             printf("OK: %llu\n", node->data);
83         }
84         else {
85             printf("NoSuchWord\n");
86         }
87     }
88 }
89 deletePatricia(root, -2);
90 }

```

laba2.c	
char toLower(char ch)	Возвращает маленький регистр буквы
void toLowercase(char str[500])	Делает каждый символ строки маленького регистра
void writePatriciaToFile(FILE file, Patricia node, int last _{bit})	Запись дерева в файл
Patricia × readPatriciaFromFile(FILE file, int check _{correct})	Чтение дерева из файла

3 Консоль

```
dmitrykochev$ cat test1.txt
+ a 1
+ aa 2
+ aaaaa 5
+ aaaaaa 6
+ aaaaaaa 7
+ aaaaaaaaa 10
+ bbccsdeqwww 11
aaaa
aaaaaaa
aaaaaaaa
aaaaaa
-aaaaaa
-aaa
-aaaaaaaaa
-aa
-aaaabbcd
bbccsdeqwww
aa
dmitrykochev$ ./lab2 <test1.txt
OK
OK
OK
OK
OK
OK
OK
NoSuchWord
OK: 7
NoSuchWord
OK: 6
OK
NoSuchWord
OK
OK
NoSuchWord
OK: 11
NoSuchWord
```


4 Тест производительности

Тесты производительности представляют из себя следующее: одни и те же данные будем вставлять, удалять и искать как в PATRICIA, так и в бинарном дереве поиска. Тестов будет три, тест характеризуется тремя числами: количество строк на вставку, поиск, и удаление из дерева. Первый тест будет содержать $5 \cdot 10^4$ строк на вставку, $5 \cdot 10^4$ на поиск и 10^4 на удаление. Второй 10^5 , 10^5 и $5 \cdot 10^4$ соответственно. Третий 10^6 , 10^6 и $5 \cdot 10^5$ соответственно. Длина строки не более 256 элементов.

```
dmitrykochev$ ./benchmark
insert to patricia: 1.4680
insert to bin tree: 1.4930
search in patricia: 1.1300
search in bin tree: 1.2480
delete in patricia: 0.2590
delete in bin tree: 0.2490
dmitrykochev$ ./benchmark
insert to patricia: 2.8980
insert to bin tree: 2.9380
search in patricia: 2.4210
search in bin tree: 2.5560
delete in patricia: 1.2850
delete in bin tree: 1.2910
dmitrykochev$ ./benchmark
insert to patricia: 31.3310
insert to bin tree: 33.6900
search in patricia: 25.4410
search in bin tree: 28.3910
delete in patricia: 13.2860
delete in bin tree: 14.5460
```

Как можно заметить, при увеличении числа строк PATRICIA становится заметно быстрее бинарного дерева.

Действительно, как говорилось выше, поиск в PATRICIA выполняется за $\Theta(h)$, в то же время в бинарном дереве поиска за $\Theta(h \cdot m)$, где m - максимальная длина строки. Для вставки ситуация аналогична. Удаление у патриции работает за $\Theta(h)$, а в дереве, из-за использования строк вместо чисел, сложность будет равна $\Theta(h \cdot m)$. Мы видим, что PATRICIA работает бинарного дерева даже с маленькой длинной строки. Это соответствует нашим расчетам.

5 Выводы

Выполнив первую лабораторную работу по курсу «Дискретный анализ», я научился писать и тестировать структуру данных PATRICIA. Безусловно, это дало мне ценный опыт в написании структуры данных своими руками с нуля. Во время выполнения я сталкивался с огромным количеством проблем. В итоге я был удивлен, какая PATRICIA удобная, быстрая и изящная структура данных, позволяющая работать со строками как с числами. После данной лабораторной работы я убедился, что могу реализовывать даже такие сложные деревья, это придает мне уверенности в себе.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* -- Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. -- 1296 с. (ISBN 5-8459-0857-4 (рус.))
- [2] Dinesh P. Mehta, Sartaj Sahni, *Handbook of DATA STRUCTURES and APPLICATIONS.*